

Network Intrusion Detection

A machine-learning IDS that reads a network flow's numbers and says normal or intrusion, and which attack family, trained on CICIoT2023: real traffic from 105 IoT devices, captured in 2023.

46.7 M

flows in the dataset

11

models compared, all from the syllabus

95.95%

attacks caught (deployed model)

8.52%

normal flows flagged

Anshuman Atrey · 150096724029

B.Tech CSE 2024-28 · Semester V · Machine Learning Fundamentals

The problem and the data

Case Study 148. Modern networks generate large volumes of traffic and detecting malicious activity manually is difficult. Build an ML-based IDS that separates normal and suspicious traffic, compare algorithms, and deploy a prototype.

Dataset: CICIoT2023 (Canadian Institute for Cybersecurity, 2023)

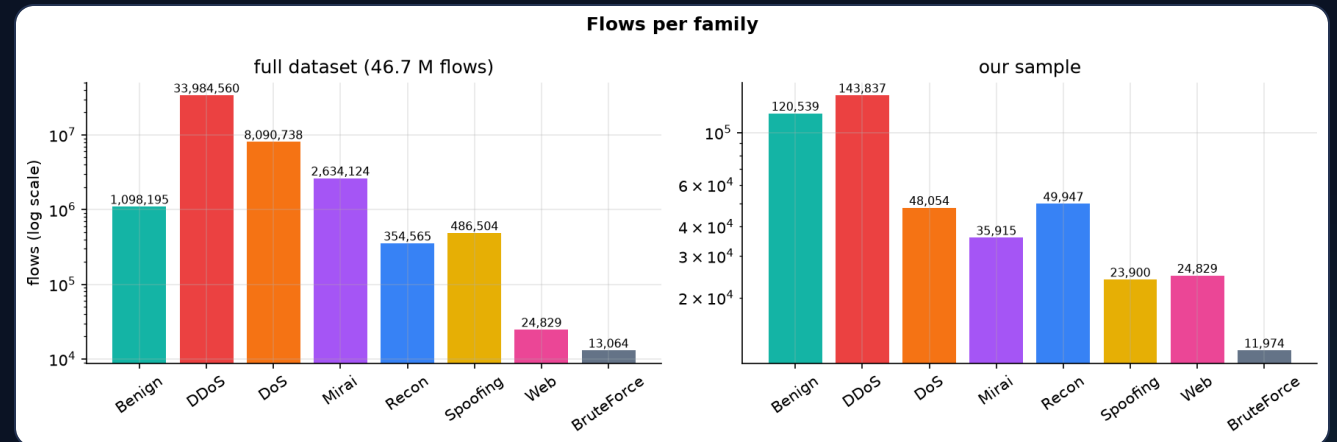
105 real IoT devices, 33 attacks in 7 families: DDoS, DoS, Mirai botnet, Recon, Spoofing, Web, BruteForce. Recent and realistic, unlike the 1999 KDD data most courses use.

A fair sample

46,686,579 flows in 169 CSV parts. Kept every rare attack and up to 12,000 of each big one, plus 120,000 benign: 458,995 flows, 0 duplicates dropped.

Fair test

stratified 75 / 25 split: 344,246 train, 114,749 test flows, 45 features each. All cleaning fitted on train only.



What the traffic looks like

Floods are loud

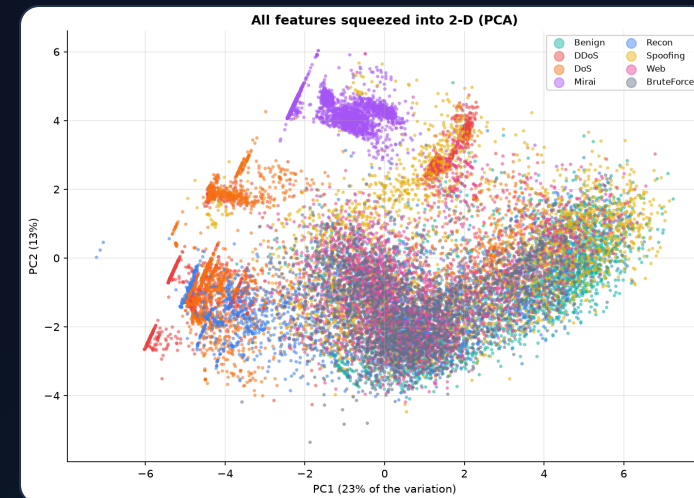
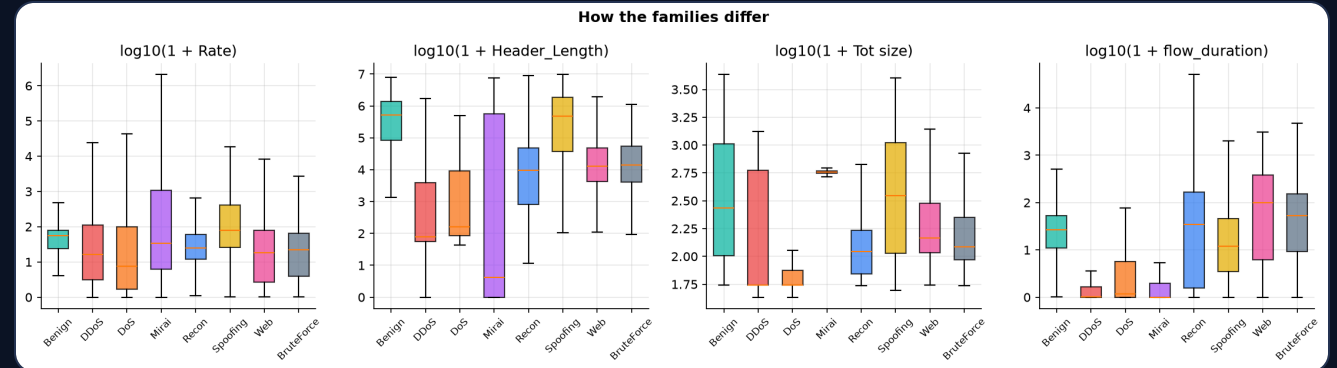
very high packet rate, tiny packets, short flows. Easy to separate.

Web attacks and brute force look like browsing

they overlap benign traffic in every view, so they will be the hard ones.

Many features say the same thing

Rate = Srate, IPv = LLC, Std $\approx$ Radius: harmless for trees, but naive bayes double-counts them.



The 99% trap: one column that knows the answer

IAT alone: 87.7% naming all 34 labels

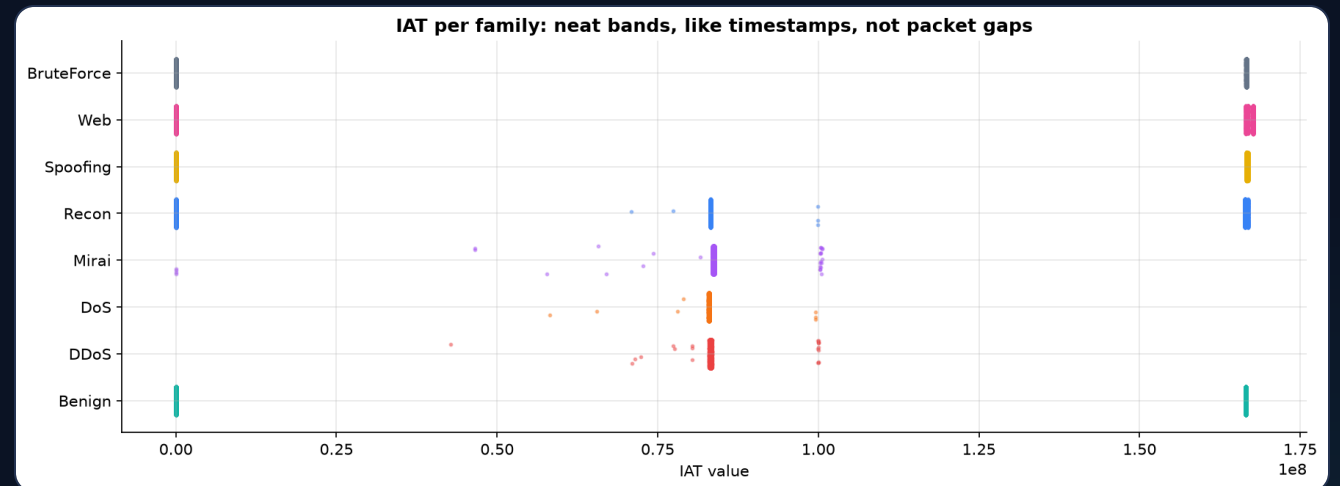
all 45 real traffic features together: 74.9%. One column cannot know more than all the traffic.

It is a clock, not a signal

IAT tracks *when* in the capture a flow happened, and each attack ran in its own time window. So it identifies the recording session, and the session gives away the attack.

Dropped from every model

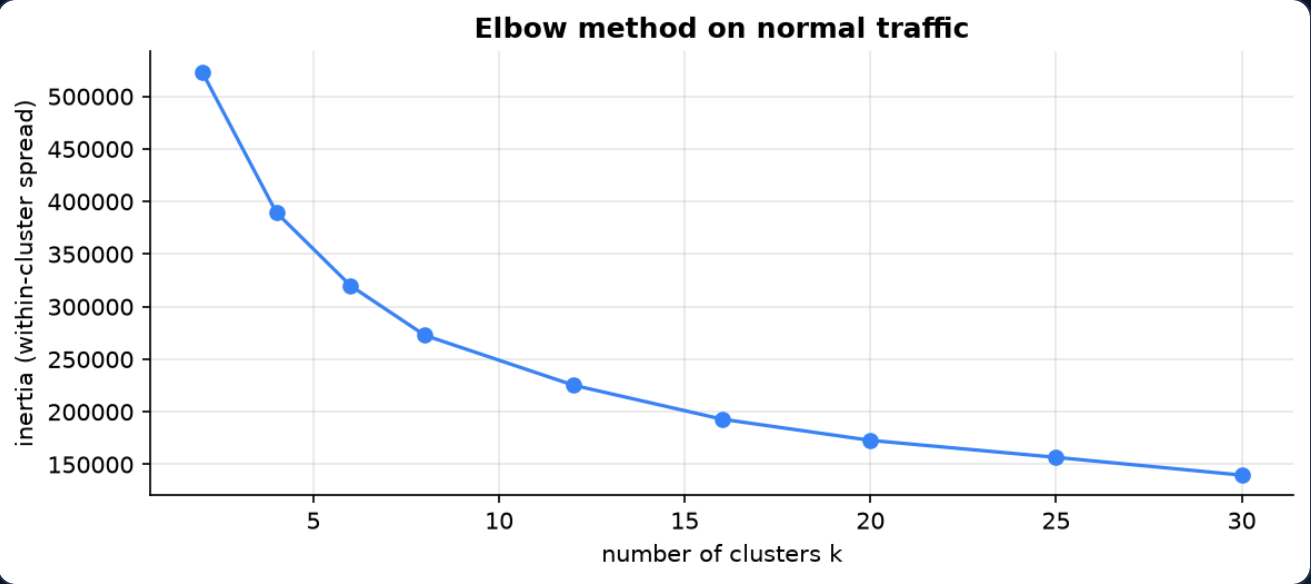
on a real network a new attack happens at a new time. Many published CICIoT2023 results are 99%+ partly because of this. Our numbers are lower and honest.



Every model from the syllabus

MODULE	MODELS
V · Classification	Logistic Regression, KNN, Decision Tree
VI · Evaluation	train / test split, 5-fold cross-validation, accuracy, precision, recall, F1, confusion matrix
VII · Unsupervised	K-Means as a detector trained on normal traffic only (k = 20 by the elbow method) · hierarchical clustering of the 34 attack types
VIII · PCA & Ensembles	PCA (20 components keep 95%) · Random Forest, Bagging, AdaBoost, Gradient Boosting
IX · Neural networks	Perceptron, MLP neural network · model saved and deployed
Case study only	Naive Bayes (asked for by Case Study 148, not in the syllabus)

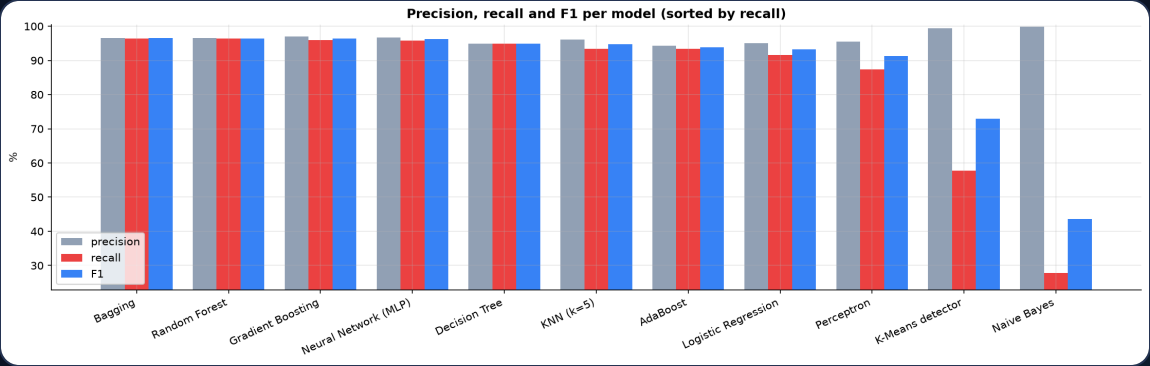
The case study asks for 6 algorithms; all 11 classifiers taught in the course run on the same training flows and the same locked test flows.



11 models, same locked test set

MODEL	MODULE	RECALL	PRECISION	F1	MISSED ATTACKS	FALSE ALARMS	MB
Bagging	VIII	96.35%	96.60%	96.48%	3,086	2,869	22.7
Random Forest	VIII	96.35%	96.50%	96.43%	3,088	2,954	106.5
Gradient Boosting	VIII	95.95%	96.93%	96.44%	3,430	2,568	0.5
Neural Network (MLP)	IX	95.85%	96.67%	96.26%	3,511	2,793	0.1
Decision Tree	V	94.91%	94.83%	94.87%	4,308	4,377	0.3
KNN (k=5)	V	93.37%	96.04%	94.68%	5,612	3,259	5.6
AdaBoost	VIII	93.36%	94.29%	93.82%	5,620	4,784	0.0
Logistic Regression	V	91.55%	95.07%	93.28%	7,148	4,016	0.0
Perceptron	IX	87.37%	95.46%	91.23%	10,689	3,519	0.0
K-Means detector	VII	57.69%	99.40%	73.01%	35,799	293	0.1
Naive Bayes	case study	27.89%	99.86%	43.60%	61,018	33	0.0

Sorted by recall, the IDS number: of the real attacks, how many it flagged. 114,749 test flows, alarm at 50%.



The models' outputs side by side

Same flows, every model

each cell: share of that family's test flows handled correctly.

Floods are easy for everyone

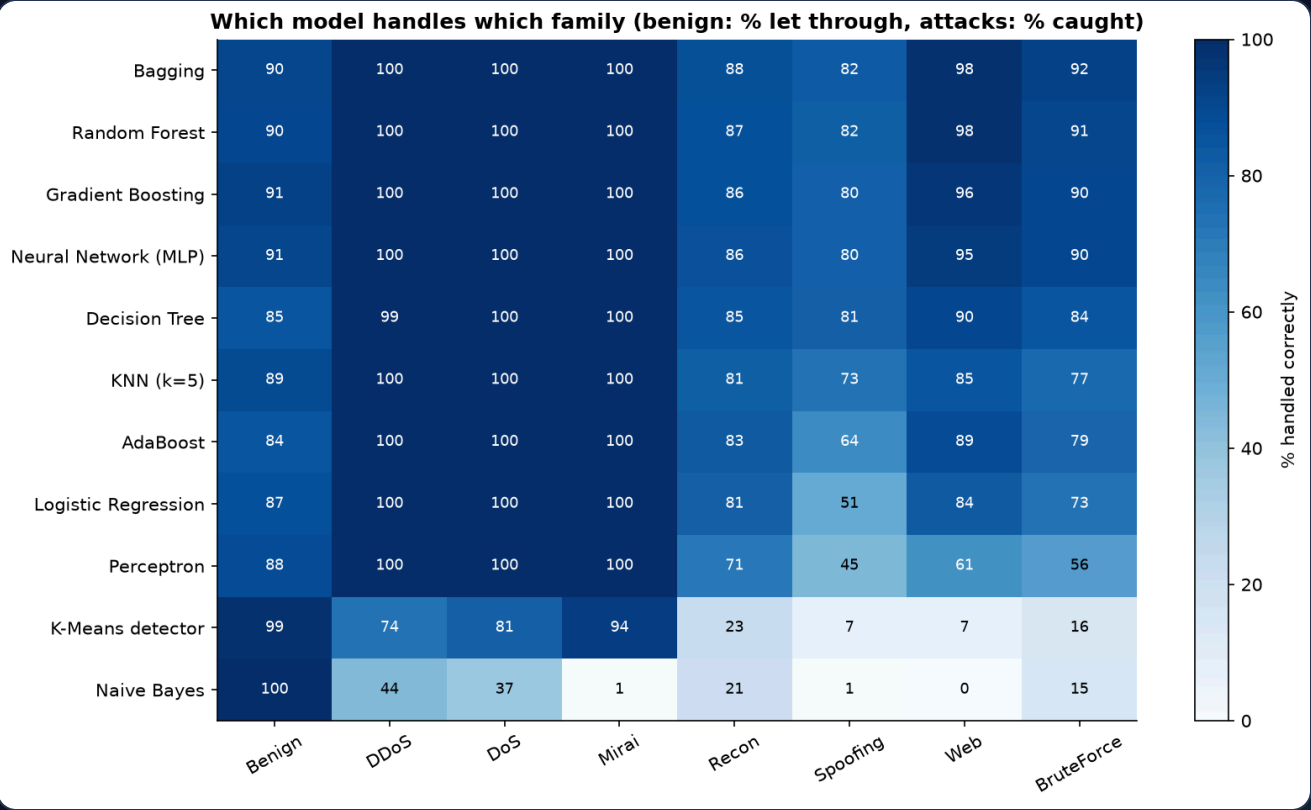
DDoS, DoS and Mirai are caught by nearly every model.

The quiet attacks split the models

Naive Bayes: Web only 0% · K-Means detector: Spoofing only 7% ·
Perceptron: Spoofing only 45%

Trees agree with each other

random forest, bagging and boosting give the same verdict on
almost every flow, so they also share their mistakes.



Where the mistakes are

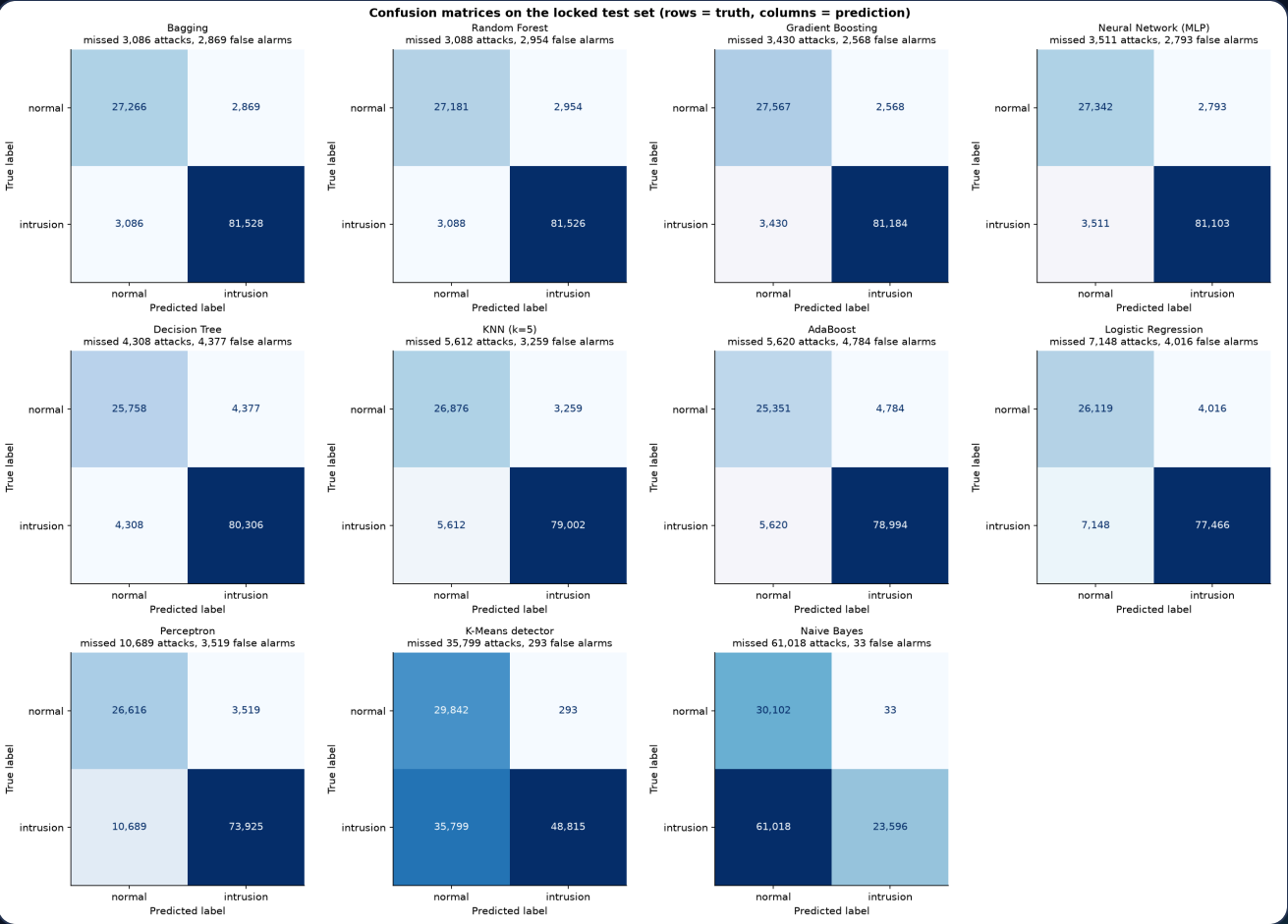
Highest recall: Bagging

misses 3,086 attacks, 2,869 false alarms.

Not a lucky split

5-fold cross-validation on the training flows:

MODEL	RECALL (5 FOLDS)	F1 (5 FOLDS)
Gradient Boosting	95.84% ± 0.11	96.36% ± 0.08
Random Forest	96.17% ± 0.08	96.33% ± 0.06



Which features and which attacks

Most important features

`rst_count`, `urg_count`, `Number`, `flow_duration`, `Header_Length`, `Rate` : shuffling them hurts the model most (permutation importance).

Family model: 84.0% accuracy, macro F1 77.7%

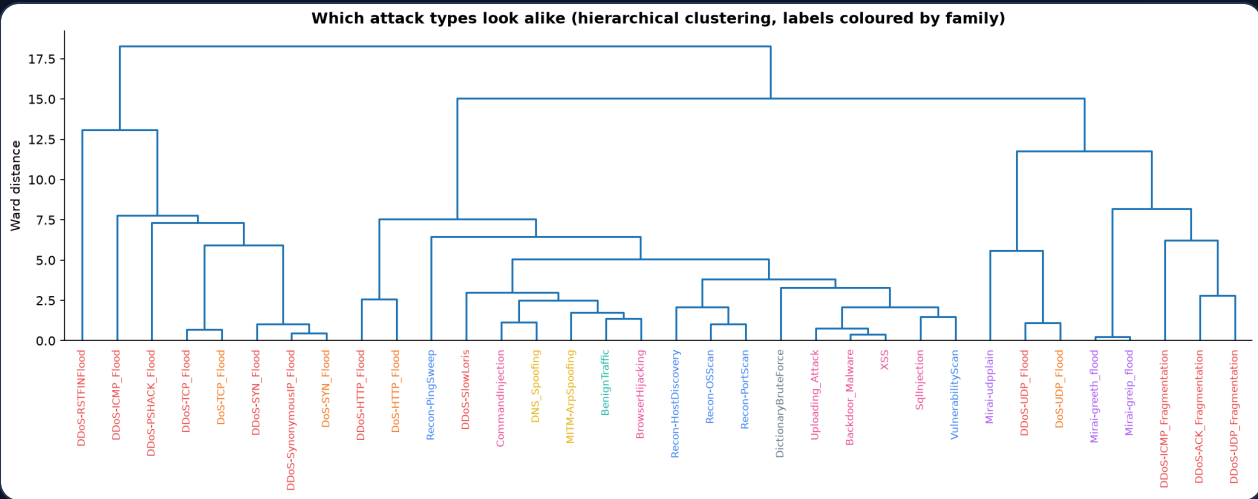
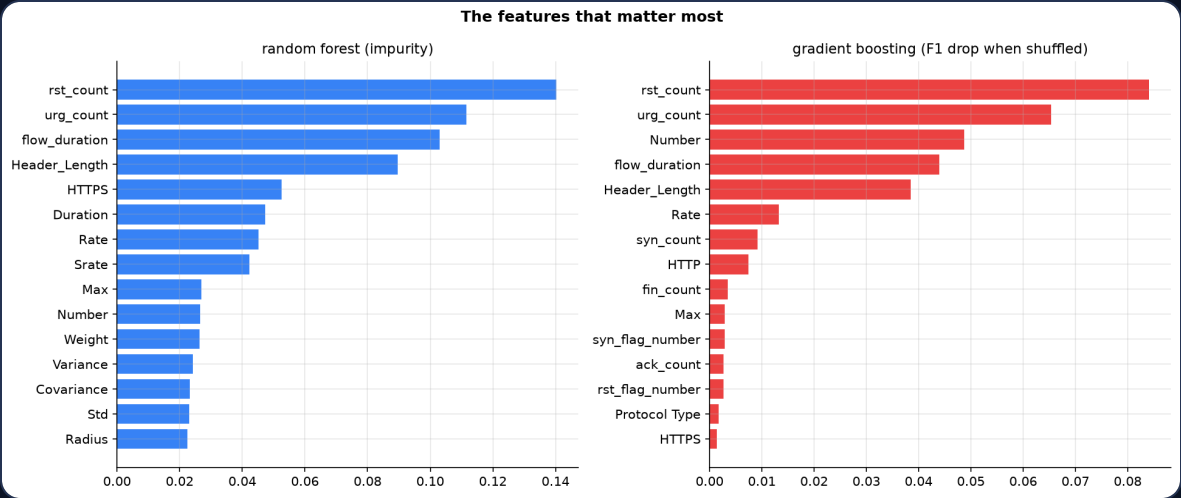
8 classes: benign + 7 attack families, gradient boosting with balanced class weights.

Hierarchical clustering

the 34 labels by their average flow: types that merge low on the tree look alike to every model.

Hardest families

Spoofing (65.8% recall), Recon (67.6% recall), BruteForce (70.9% recall): they look like ordinary traffic.



Class imbalance and unseen attacks

Accuracy lies

always answering “intrusion” already scores high on this data, so we rank on recall and F1.

Weighting moves recall, it does not add it

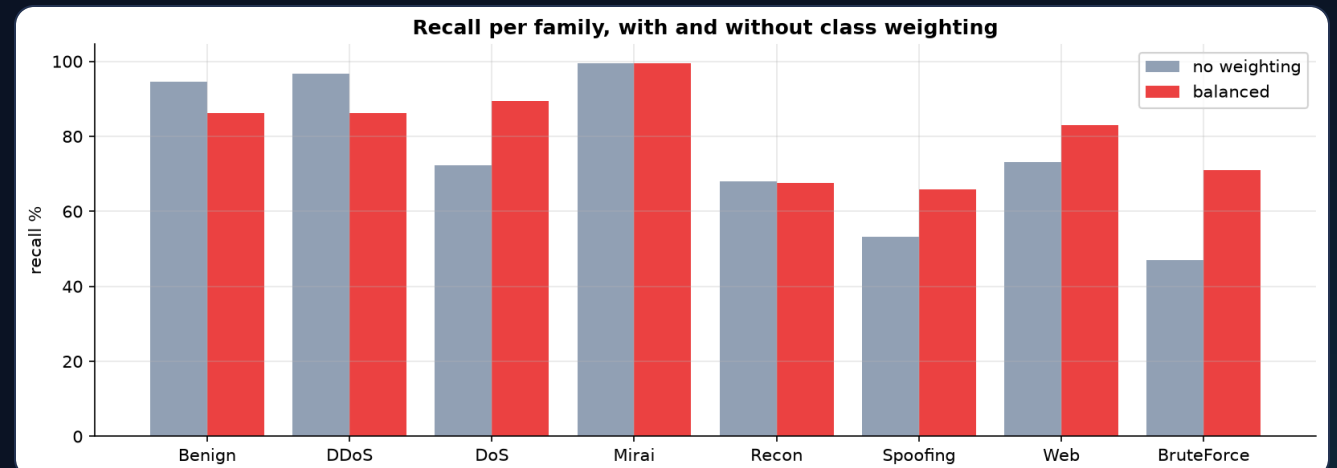
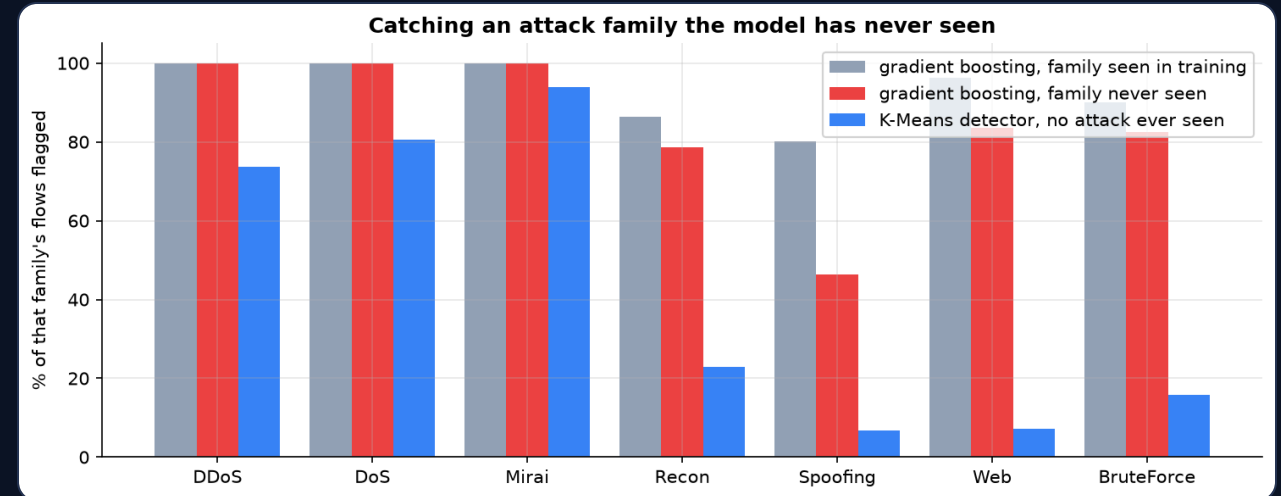
balanced class weights: brute force 47% → 71%, web 73% → 83%, but DDoS 97% → 86%; macro F1 flat (77.8% vs 77.7%).

Precision depends on how rare attacks are

50.0% attacks → 91.8% precision · 1.0% attacks → 10.2% precision · 0.1% attacks → 1.1% precision

Never-seen families

trained without a family, then tested on it: Mirai still 100.0% caught, Spoofing only 46.5%. The K-Means detector, which never saw any attack, catches floods but not the quiet ones.



The prototype: Flow Sentinel (Next.js)

Live: walrus-flow-sentinel.streamlit.app, in the Walrus Securitas look (white and orange), hosted on Streamlit Community Cloud.

Detector

pick one of 300 real test flows on the traffic tape, or type in its features: normal / intrusion, the intrusion score, the alarm threshold, and all 8 family probabilities.

Scan a file

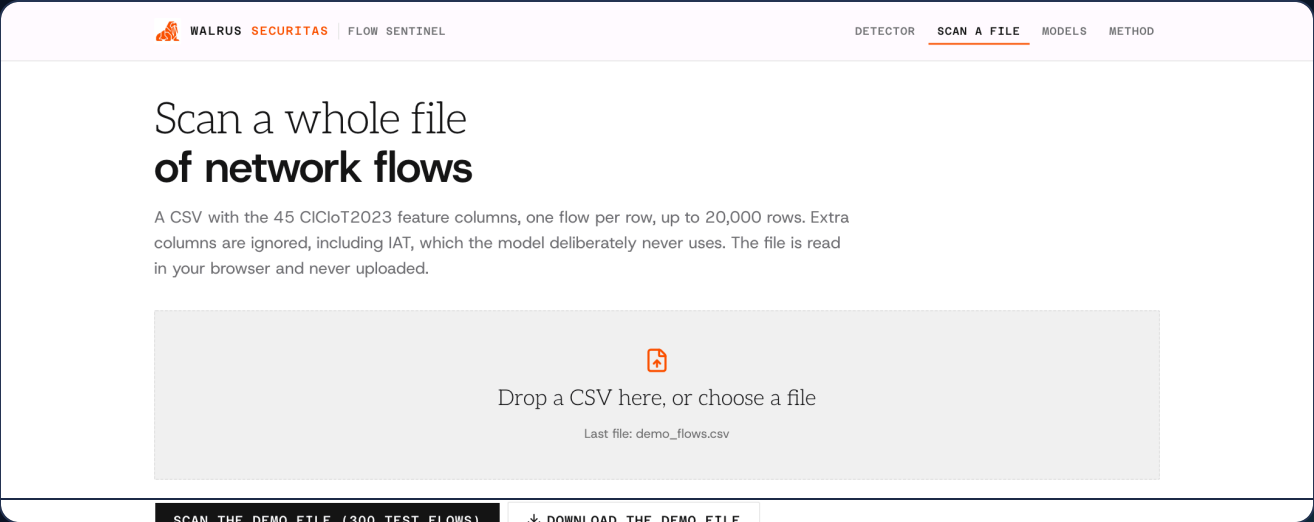
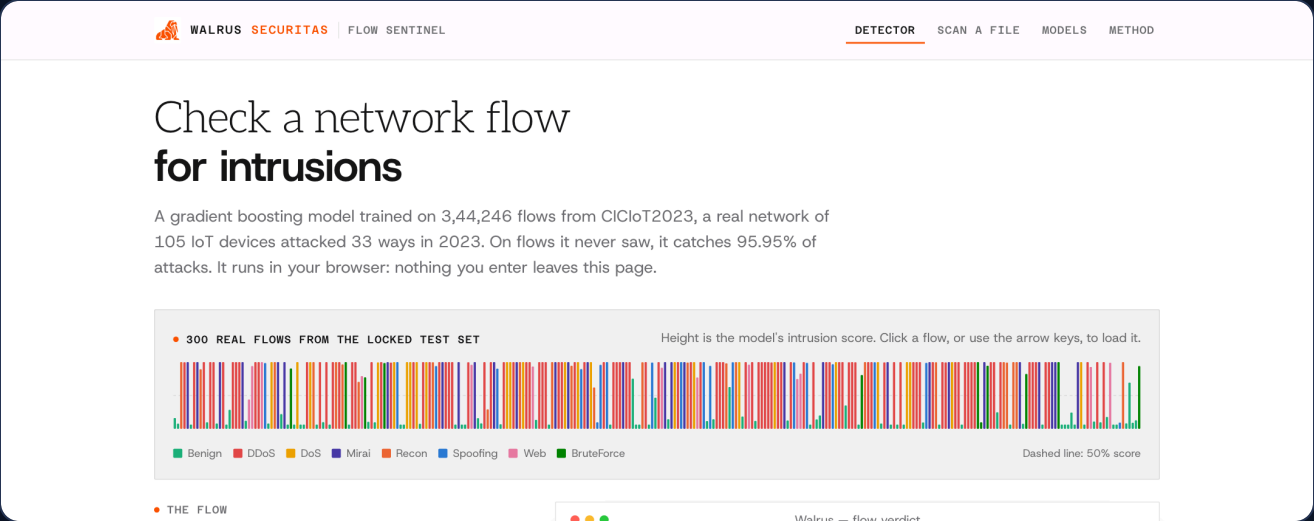
drop a CSV of up to 20,000 flows: verdicts, a sortable table, a download. Missing columns, text, negative or infinite values are caught.

Model comparison and method

all 11 models, the per-family heatmap, the unseen-attack test, the IAT trap and the six answers, every number read from the notebook's results.

Runs in the browser

gradient boosting (95.95% recall) exported to JSON; a test checks the browser's predictions match sklearn's on 300 flows. Next.js 16, Tailwind, shadcn/ui, Recharts.



Answers and limitations

QUESTION	ANSWER
Can ML separate normal and malicious traffic?	Yes: 95.95% of attacks caught, 8.52% of normal flows flagged.
Which features matter most?	rst_count, urg_count, Number, flow_duration
Highest recall?	Bagging (96.35%)
Hardest attack categories?	Spoofing, Recon, BruteForce
Effect of class imbalance?	accuracy misleads, rare families get ignored without weighting, precision falls when attacks are rare
Previously unseen traffic?	partly: 100.0% for Mirai, 46.5% for Spoofing

Limitations

lab traffic from one IoT testbed · features are averaged windows, not single packets · benign share set by our sampling · new attack families are only partly caught · one layer of defence, not a guarantee.

Next

test on another network's traffic (cross-dataset), add an anomaly detector for unseen attacks.